```
    when "other" then "bg-red-600"
    when "purchased" then "bg-green-600"
    when "held" then "bg-green-600"
    end
  end
end
```

Using Stimulus to Take Action

All that code gets us the display and the data state in our initial HTML rendered on the server. Now we need to implement the interactivity.

The first interactive bit is that changing the pull-down menu for the number of tickets being purchased changes the state of tickets in the display and requires the entire display to be redrawn. We're going to do this as a regular form submission that uses a Turbo Stream to update the page.

If you look up at the earlier `_venue_header` partial view file, the form is already declared inside a Stimulus controller named `form` and is the `form` target of that controller. The form itself sets a URL of `tickets_to_buy_path` and the select box sets a Stimulus action of `"change->form#submit"`, meaning that when the select box changes it will call a `submit` method on the Stimulus controller.

The controller itself is very generic:

appendix_hotwire/01/app/javascript/controllers/form_controller.js

```
import { Controller } from "@hotwired/stimulus"

export default class FormController extends Controller {
  static targets = ["form"]

  submit() {
    this.formTarget.requestSubmit()
  }
}
```

All it does is allow Stimulus to submit a form (it's a simplification of the controller you saw in Chapter 8, *Talking to the Server*).